

Condition variables

Alessia Smeralda Brunello

Matricola: 544964

Hands-On 13

Indice

1	Introduzione	2
1.1	Esercizio 1	2
1.2	Esercizio 2	2
2	Definizione del problema	2
2.1	Esercizio 1: attesa della terminazione del thread figlio	2
2.2	Esercizio 2: barriera per quattro thread	3
3	Metodologia	3
3.1	Esercizio 1	3
3.2	Esercizio 2	4
4	Risultati e output con osservazioni	6
4.1	Compilazione	6
4.2	Esercizio 1: output e osservazioni	6
4.3	Esercizio 2: output e osservazioni	8
5	Conclusioni	9

1 Introduzione

L'Hands-On 13 ha come obiettivo lo studio e l'utilizzo delle *condition variables* della libreria POSIX thread. Le condition variables sono uno strumento di sincronizzazione che permette a un thread di sospendere la propria esecuzione fino al verificarsi di una determinata condizione logica condivisa con altri thread.

Il loro utilizzo consente di evitare l'attesa attiva, cioè il controllo continuo di una variabile in un ciclo, riducendo il consumo inutile di CPU. Una condition variable viene normalmente utilizzata insieme a un mutex, poiché la condizione da controllare è rappresentata da una variabile condivisa che deve essere letta e modificata in modo sicuro.

La traccia mostra il pattern fondamentale da applicare: proteggere una condizione logica con un mutex, sospendere il thread tramite `pthread_cond_wait()` e risvegliare uno o più thread tramite `pthread_cond_signal()` o `pthread_cond_broadcast()`.

Il lavoro è stato organizzato in modo modulare, separando le definizioni nei file header, le implementazioni nei file sorgente e i programmi principali in file dedicati. La compilazione è stata automatizzata tramite `Makefile`, così da ottenere due eseguibili distinti, uno per ciascun esercizio.

1.1 Esercizio 1

Nel primo esercizio si richiede di supporre che la funzione `pthread_join()` non esista. Il problema consiste quindi nel far attendere il thread padre fino alla terminazione del thread figlio, utilizzando esclusivamente mutex e condition variables. La traccia specifica inoltre che non è necessario acquisire il valore di ritorno del thread figlio.

1.2 Esercizio 2

Nel secondo esercizio si richiede di realizzare, sempre attraverso condition variables, un meccanismo di barriera che permetta a quattro thread di partire contemporaneamente. In questo contesto, partire contemporaneamente significa che nessun thread può superare la barriera finché tutti e quattro non l'hanno raggiunta.

2 Definizione del problema

2.1 Esercizio 1: attesa della terminazione del thread figlio

Nel primo esercizio il problema riguarda la sincronizzazione tra due thread:

- **il thread padre:** crea il thread figlio e deve attendere la sua terminazione;
- **il thread figlio:** esegue un lavoro e successivamente segnala al padre di aver terminato.

In condizioni normali, tale sincronizzazione verrebbe ottenuta tramite la funzione `pthread_join()`. Tuttavia, poiché la traccia richiede di non utilizzare tale funzione, è necessario introdurre un meccanismo alternativo basato su una condition variable.

La soluzione si basa su una variabile condivisa, chiamata **terminato**, che rappresenta lo stato del thread figlio. La condizione logica da verificare è la seguente:

```
terminato == 1
```

Finché questa condizione non è vera, il padre deve rimanere sospeso. Quando il figlio termina il proprio lavoro, modifica la variabile condivisa e segnala la condition variable, permettendo al padre di riprendere l'esecuzione.

La correttezza della soluzione dipende dal fatto che la variabile **terminato** viene sempre letta e modificata all'interno di una sezione critica protetta da mutex.

2.2 Esercizio 2: barriera per quattro thread

Nel secondo esercizio il problema consiste nel coordinare quattro thread affinché nessuno di essi possa proseguire oltre un certo punto del programma prima che tutti gli altri abbiano raggiunto lo stesso punto.

Questo comportamento viene realizzato tramite una barriera. Ogni thread, quando raggiunge la barriera, incrementa un contatore condiviso. Se non è l'ultimo thread ad arrivare, si sospende sulla condition variable. L'ultimo thread, invece, apre la barriera e risveglia tutti i thread in attesa.

La condizione logica della barriera è:

```
arrivati == total
```

dove **arrivati** indica quanti thread hanno raggiunto la barriera e **total** indica il numero totale di thread da attendere, pari a 4.

Anche in questo caso la correttezza della soluzione dipende dalla protezione delle variabili condivise tramite mutex. In particolare, l'incremento del contatore **arrivati** deve avvenire in mutua esclusione, per evitare condizioni di race.

3 Metodologia

3.1 Esercizio 1

La soluzione del primo esercizio è stata organizzata in modo modulare attraverso i file `join.h`, `join.c` e `join_main.c`. L'idea principale è sostituire l'attesa realizzata normalmente con `textttpthread_join()` con una condition variable associata a una variabile condivisa.

La struttura dati utilizzata per rappresentare questo meccanismo contiene un mutex, una condition variable e la variabile intera **terminato**:

```
1     typedef struct {
2         pthread_mutex_t mutex;
3         pthread_cond_t cond;
4         int terminado;
5     } join_t;
```

Il padre utilizza una funzione di attesa che controlla la variabile `terminato`. Il nucleo della sincronizzazione è rappresentato dalle seguenti istruzioni:

```
1 pthread_mutex_lock(&join->mutex);
2
3 while (!join->terminato) {
4     pthread_cond_wait(&join->cond, &join->mutex);
5 }
6
7 pthread_mutex_unlock(&join->mutex);
```

La chiamata a `pthread_cond_wait()` sospende il thread padre e rilascia temporaneamente il mutex. Quando il thread viene risvegliato, il mutex viene riacquisito automaticamente prima di uscire dalla funzione.

L'utilizzo del ciclo `while` è fondamentale: il risveglio da una condition variable non implica necessariamente che la condizione logica sia vera. Per questo motivo, dopo ogni risveglio, la condizione deve essere controllata nuovamente.

Il thread figlio, dopo aver completato il proprio lavoro, aggiorna la variabile condivisa e segnala la condition variable:

```
1 join->terminato = 1;
2 pthread_cond_signal(&join->cond);
```

L'ordine delle due operazioni è importante: prima viene resa vera la condizione logica, poi viene inviato il segnale. In questo modo, quando il padre viene risvegliato, può verificare che la condizione richiesta sia effettivamente soddisfatta.

Nel file `join_main.c`, il comportamento del thread figlio è stato reso più osservabile tramite alcune stampe e una simulazione del lavoro svolto:

```
1 printf("[Figlio %d] thread avviato\n", args->id);
2
3 simula_lavoro_figlio(args->id);
4
5 printf("[Figlio %d] lavoro terminato\n", args->id);
6
7 join_signal_done(args->join);
```

Queste istruzioni permettono di visualizzare l'ordine degli eventi: il thread figlio parte, esegue alcune fasi di lavoro, termina e solo successivamente segnala il padre.

Poiché la traccia richiede di non usare `pthread_join()` per attendere il figlio, il thread figlio viene creato in modalità *detached*. In questo modo, le sue risorse vengono rilasciate automaticamente alla terminazione.

3.2 Esercizio 2

La soluzione del secondo esercizio è stata organizzata attraverso i file `barrier.h`, `barrier.c` e `barrier_main.c`. L'obiettivo è realizzare una barriera per quattro thread usando mutex e condition variable.

La struttura dati della barriera contiene le informazioni necessarie per gestire l'attesa dei thread:

```
1     typedef struct {
2         pthread_mutex_t mutex;
3         pthread_cond_t cond;
4         int arrivati;
5         int total;
6         int aperta;
7     } barrier_t;
```

La variabile **arrivati** conta quanti thread hanno raggiunto la barriera, mentre **total** rappresenta il numero totale di thread da attendere. Il flag **aperta** indica se la barriera è stata sbloccata.

La funzione principale della soluzione è **barrier_wait()**. Quando un thread raggiunge la barriera, acquisisce il mutex e incrementa il contatore condiviso:

```
1     pthread_mutex_lock(&barrier->mutex);
2
3     barrier->arrivati++;
4
5     printf("[Barriera] thread %d arrivato alla barriera (%d/%d)\n",
6           id_thread, barrier->arrivati, barrier->total);
```

Il mutex è necessario perché la variabile **arrivati** è condivisa da tutti i thread. Senza mutua esclusione, due thread potrebbero modificarla contemporaneamente, producendo un valore non corretto.

Dopo l'incremento, il thread verifica se è l'ultimo ad arrivare. Se il numero di thread arrivati coincide con il numero totale di thread attesi, la barriera viene aperta:

```
1     if (barrier->arrivati == barrier->total) {
2         printf("[Thread %d] sono l'ultimo; sblocco la barriera\n",
3               id_thread);
4
5         barrier->aperta = 1;
6         pthread_cond_broadcast(&barrier->cond);
7     }
```

In questo caso viene utilizzata `pthread_cond_broadcast()` perché non deve essere risvegliato un solo thread, ma tutti i thread sospesi sulla barriera. L'ultimo thread ha quindi un ruolo particolare: riconosce il completamento della barriera, modifica lo stato condiviso e risveglia gli altri thread.

I thread che non sono gli ultimi rimangono invece in attesa:

```
1     else {
2         while (!barrier->aperta) {
3             pthread_cond_wait(&barrier->cond, &barrier->mutex);
4         }
5
6         printf("[Thread %d] risvegliato; la barriera e' aperta\n",
7               id_thread);
8     }
```

Anche qui il ciclo `while` serve a ricontrollare la condizione prima di proseguire.

Nel programma principale, ogni thread esegue una fase iniziale, raggiunge la barriera e prosegue solo dopo lo sblocco:

```
1     printf("[THREAD %d] sono pronto, ma devo aspettare gli altri\n", args->
2           id);
3
4     barrier_wait(args->barrier, args->id);
```

```
4
5     printf("[THREAD %d] partito\n", args->id);
```

Queste stampe permettono di verificare sperimentalmente il comportamento della barriera: i thread possono arrivare in momenti diversi, ma nessuno prosegue prima dell'apertura della barriera.

Nel `main` è presente anche `pthread_join()`, utilizzato esclusivamente per evitare che il processo principale termini prima dei thread creati. Questo non modifica la soluzione del problema, poiché la sincronizzazione della barriera è realizzata tramite condition variables.

4 Risultati e output con osservazioni

4.1 Compilazione

La compilazione è stata gestita tramite `Makefile`. Il progetto produce due eseguibili distinti:

- `join_app`, relativo all'esercizio 1;
- `barriera_app`, relativo all'esercizio 2.

I principali target usati per l'esecuzione sono:

- `run-join`, per eseguire il primo esercizio;
- `run-barrier`, per eseguire il secondo esercizio.

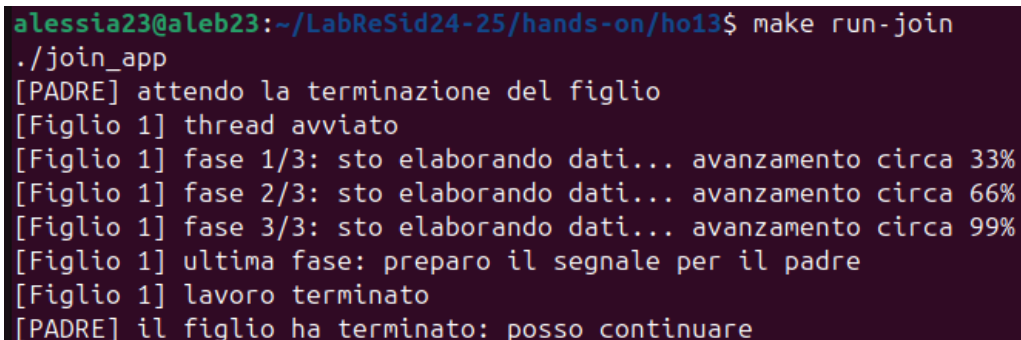
La separazione in due eseguibili consente di testare in modo indipendente le due soluzioni.

4.2 Esercizio 1: output e osservazioni

Eseguendo il primo esercizio tramite:

```
1     make run-join
```

si ottiene il seguente output:



```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho13$ make run-join
./join_app
[PADRE] attendo la terminazione del figlio
[Figlio 1] thread avviato
[Figlio 1] fase 1/3: sto elaborando dati... avanzamento circa 33%
[Figlio 1] fase 2/3: sto elaborando dati... avanzamento circa 66%
[Figlio 1] fase 3/3: sto elaborando dati... avanzamento circa 99%
[Figlio 1] ultima fase: preparo il segnale per il padre
[Figlio 1] lavoro terminato
[PADRE] il figlio ha terminato: posso continuare
```

L'output mostra che il padre si mette in attesa della terminazione del figlio. Il figlio, nel frattempo, esegue alcune fasi di lavoro simulate e stampa il proprio avanzamento. Solo dopo aver completato il lavoro, il figlio segnala la condition variable.

Il risultato conferma che il padre non prosegue prima della terminazione del figlio. La condition variable realizza quindi un comportamento analogo a quello di `pthread_join()` per quanto riguarda l'attesa, ma senza recuperare alcun valore di ritorno dal thread figlio, come richiesto dalla traccia.

Dal punto di vista della sincronizzazione, il comportamento corretto è garantito da tre elementi:

- la variabile condivisa **terminato**;
- il mutex, che protegge l'accesso a tale variabile;
- la condition variable, che permette al padre di sospendersi e di essere risvegliato dal figlio.

4.3 Esercizio 2: output e osservazioni

Eseguendo il secondo esercizio tramite:

```
1 make run-barrier
```

si ottiene il seguente output:

```
alessia23@aleb23:~/LabReSid24-25/hands-on/ho13$ make run-barrier
./barriera_app
[MAIN] creo 4 thread e inizializzo la barriera
[THREAD 1] inizializzazione locale in corso
[THREAD 2] inizializzazione locale in corso
[THREAD 3] inizializzazione locale in corso
[THREAD 4] inizializzazione locale in corso

[THREAD 1] sono pronto, ma devo aspettare gli altri
[Barriera] thread 1 arrivato alla barriera(1/4)

[THREAD 2] sono pronto, ma devo aspettare gli altri
[Barriera] thread 2 arrivato alla barriera(2/4)

[THREAD 3] sono pronto, ma devo aspettare gli altri
[Barriera] thread 3 arrivato alla barriera(3/4)

[THREAD 4] sono pronto, ma devo aspettare gli altri
[Barriera] thread 4 arrivato alla barriera(4/4)

[Thread 4] sono l'ultimo; sblocco la barriera
[THREAD 4] partito

[Thread 2] risvegliato; la barriera è aperta
[THREAD 2] partito

[Thread 3] risvegliato; la barriera è aperta
[THREAD 3] partito

[Thread 1] risvegliato; la barriera è aperta
[THREAD 1] partito
[THREAD 4] lavoro completato
[THREAD 2] lavoro completato
[THREAD 3] lavoro completato
[THREAD 1] lavoro completato
[MAIN] tutti i thread hanno terminato
```

L'output mostra che i thread arrivano alla barriera in momenti diversi. Questo comportamento è reso evidente dalle stampe e dai ritardi introdotti nella fase iniziale di ciascun thread.

I primi 3 thread raggiungono la barriera, incrementano il contatore e poi si sospendono, poiché il numero di thread arrivati non è ancora pari al totale. Quando arriva il quarto thread, il contatore raggiunge il valore 4/4. Il quarto thread riconosce di essere l'ultimo, stampa il messaggio corrispondente e sblocca la barriera tramite `pthread_cond_broadcast()`.

Dopo lo sblocco, tutti i thread possono proseguire. L'ordine con cui i thread stampano i messaggi di risveglio e di partenza non è deterministico, perché dipende dallo scheduler del sistema operativo.

La proprietà importante è che nessun thread stampi **partito** prima che la barriera sia stata aperta.

Il risultato conferma quindi che la barriera implementata rispetta la specifica della traccia: i quattro thread attendono tutti lo stesso punto di sincronizzazione prima di continuare l'esecuzione.

5 Conclusioni

L'Hands-On ha permesso di applicare le condition variables a due problemi classici di sincronizzazione tra thread: l'attesa della terminazione di un thread figlio e la realizzazione di una barriera.

Nel primo esercizio è stato riprodotto il comportamento principale di `pthread_join()` utilizzando una variabile condivisa, un mutex e una condition variable. Nel secondo esercizio, invece, è stata implementata una barriera per quattro thread, in cui l'ultimo thread ad arrivare sblocca tutti gli altri tramite `pthread_cond_broadcast()`.

In entrambi i casi, la correttezza della soluzione dipende dall'associazione tra condition variable e condizione logica esplicita. Il mutex protegge lo stato condiviso, mentre la condition variable consente ai thread di sospendersi e risvegliarsi senza ricorrere all'attesa attiva.

L'esecuzione dei programmi conferma il comportamento atteso: nel primo caso il padre prosegue solo dopo la terminazione del figlio, mentre nel secondo caso nessun thread supera la barriera prima che tutti e quattro l'abbiano raggiunta.